Array & String Problems Cheatsheet

1. Core Concepts

Prefix Sum

Idea: Store cumulative sum to answer range queries in O(1).

Formula: $\text{prefix}[i] = \sum_{j=0}^{i} \text{arr}[j]$

Range Sum: $\text{sum}(L, R) = \text{prefix}[R] - \text{prefix}[L-1]$

```
// Build prefix sum
prefix[0] = arr[0];
for(int i = 1; i < n; i++) {
    prefix[i] = prefix[i-1] + arr[i];
}

// Query [L, R]
int sum = (L == 0) ? prefix[R]
    : prefix[R] - prefix[L-1];
```

Time: O(n) build, O(1) query
Space: O(n)

Two Pointer Technique

When to use: Sorted arrays, finding pairs, palindromes

Pattern:

- Start: $left = 0, right = n - 1$
- Move based on condition
- Converge until $left < right$

```
int l = 0, r = n - 1;
while(l < r) {
    int sum = arr[l] + arr[r];
    if(sum == target) return true;
    else if(sum < target) l++;
    else r--;
}
```

Time: O(n) single pass
Space: O(1)

Binary Search Pattern

Template 1: Find exact match

```
int l = 0, r = n - 1;
while(l <= r) {
    int mid = l + (r - l) / 2;
    if(arr[mid] == target) return mid;
    else if(arr[mid] < target) l = mid+1;
    else r = mid - 1;
}
return -1;
```

Template 2: Find leftmost

```
int l = 0, r = n - 1;
while(l < r) {
    int mid = l + (r - l) / 2;
    if(arr[mid] >= target) r = mid;
    else l = mid + 1;
}
return arr[l] == target ? l : -1;
```

Template 3: Find rightmost

```
int l = 0, r = n - 1;
while(l < r) {
    int mid = l + (r - l + 1) / 2; // +1!
    if(arr[mid] <= target) l = mid;
    else r = mid - 1;
}
return arr[l] == target ? l : -1;
```

Time: O(log n)
Space: O(1)

HashMap Patterns

Frequency Count:

```
Map<Integer, Integer> freq = new HashMap<>();
for(int num : arr) {
    freq.put(num, freq.getOrDefault(num, 0) + 1);
}
```

Grouping:

```
Map<Key, List<Value>> groups = new HashMap<>();
for(Value v : values) {
    Key k = computeKey(v);
    groups.putIfAbsent(k, new ArrayList<>());
    groups.get(k).add(v);
}
```

Two Sum Pattern:

```
Map<Integer, Integer> seen = new HashMap<>();
for(int i = 0; i < n; i++) {
    int complement = target - nums[i];
    if(seen.containsKey(complement)) {
        return new int[]{seen.get(complement), i};
    }
    seen.put(nums[i], i);
}
```

Min Heap (PriorityQueue)

Operations:

- `offer(e)`: Add element - O(log n)
- `poll()`: Remove min - O(log n)
- `peek()`: Get min - O(1)
- `size()`: Get size - O(1)

Min Heap (default):

```
PriorityQueue<Integer> minHeap =
    new PriorityQueue<>();
```

Max Heap:

```
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>((a,b) -> b - a);
```

Custom Comparator:

```
PriorityQueue<int[]> heap =
    new PriorityQueue<>((a,b) -> a[0] - b[0]);
```

Use Cases:

- Top K elements
- Kth largest/smallest
- Merge K sorted arrays
- Median of stream

2D Array Basics

Declaration:

```
int[][] arr = new int[rows][cols];
int[][] arr = {{1,2,3}, {4,5,6}};
```

Access:

```
arr[i][j]      // Element at row i, col j
arr.length     // Number of rows
arr[i].length  // Number of cols in row i
```

Iteration:

```
// Method 1
for(int i = 0; i < arr.length; i++) {
    for(int j = 0; j < arr[i].length; j++) {
        // Process arr[i][j]
    }
}

// Method 2
for(int[] row : arr) {
    for(int val : row) {
        // Process val
    }
}
```

References in Java

Key Concept: Arrays and objects store references, not values!

```
int[] arr1 = {1, 2, 3};
int[] arr2 = arr1;  // Same reference!
arr2[0] = 99;
// arr1[0] is now 99 too!

// To copy:
int[] arr3 = Arrays.copyOf(arr1, arr1.length);
```

In Lists:

```
List<int[]> list = new ArrayList<>();
int[] arr = {1, 2};
list.add(arr);  // Stores reference

arr[0] = 99;    // Modifies array in list!
```

2. Problems & Solutions

Problem 1: Best Time to Buy & Sell Stock

Problem Statement

Given array of prices, find max profit from one transaction.
Example: `[7,1,5,3,6,4]` → 5 (buy at 1, sell at 6)

Approach: One Pass (Optimal)

Idea: Track min price seen, update max profit at each step.

```
class Solution {
    public int maxProfit(int[] prices) {
        int maxProfit = 0, minPrice = Integer.MAX_VALUE;

        for(int price : prices) {
            minPrice = Math.min(minPrice, price);
            maxProfit = Math.max(maxProfit, price -
                minPrice);
        }
        return maxProfit;
    }
}
```

Time: O(n) - single pass
Space: O(1) - two variables

Common Mistakes

× Checking all pairs ($O(n^2)$)
× Selling before buying
✓ Update minPrice first, then maxProfit

Problem 2: Contains Duplicate

Problem Statement

Return true if any value appears at least twice.
Example: `[1,2,3,1]` → true

Approach 1: HashSet (Optimal)
Idea: Use set to track seen elements.

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();
        for(int num : nums) {
            if(!seen.add(num)) return true;
        }
        return false;
    }
}
```

Time: O(n) - single pass
Space: O(n) - HashSet

Approach 2: Sorting
Idea: Sort, check adjacent elements.

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Arrays.sort(nums);
        for(int i = 0; i < nums.length - 1; i++) {
            if(nums[i] == nums[i+1]) return true;
        }
        return false;
    }
}
```

Time: O(n log n) - sorting
Space: O(log n) - sort stack

When to Use Each

- HashSet: Faster (O(n)), use by default
- Sorting: When space is extremely limited

Problem 3: Two Sum II (Sorted)

Problem Statement

Find two numbers that add to target. Array is sorted, 1-indexed.
Example: `[2,7,11,15]`, target=9 → `[1,2]`

Approach: Two Pointer (Optimal)
Idea: Use sorted property with two pointers.

```
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int left = 0, right = numbers.length - 1;

        while(left < right) {
            int sum = numbers[left] + numbers[right];
            if(sum == target) {
                return new int[]{left + 1, right + 1};
            } else if(sum < target) {
                left++;
            } else {
                right--;
            }
        }
        return new int[]{};
    }
}
```

Time: O(n) - single pass
Space: O(1) - two pointers

Common Mistakes

× Forgetting 1-indexed return
× Using HashMap (doesn't use sorted property)
✓ Move left if sum too small, right if too large

Problem 4: Three Sum

Problem Statement

Find all unique triplets that sum to zero.
Example: `[-1,0,1,2,-1,-4]` → `[[-1,-1,2], [-1,0,1]]`

Approach: Sort + Two Pointer
Idea: Fix first element, use two pointers for remaining two.

```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();

        for(int i = 0; i < nums.length - 2; i++) {
            // Skip duplicates for i
            if(i > 0 && nums[i] == nums[i-1]) continue;

            // Early termination
            if(nums[i] > 0) break;

            int left = i + 1, right = nums.length - 1;
            while(left < right) {
                int sum = nums[i] + nums[left] + nums[
                    right];

                if(sum == 0) {
                    result.add(Arrays.asList(
                        nums[i], nums[left], nums[right]))
                            ;
                    left++;
                    right--;

                    // Skip duplicates
                    while(left < right &&
                          nums[left] == nums[left-1]) left
                              ++;
                    while(left < right &&
                          nums[right] == nums[right+1])
                              right--;
                } else if(sum < 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return result;
    }
}
```

Time: $O(n^2)$ - $O(n \log n)$ sort + $O(n^2)$ loop
Space: O(1) extra (output doesn't count)

Common Mistakes

× Wrong duplicate skipping direction
× Using `while` instead of `if` for outer loop
✓ Skip duplicates AFTER finding match
✓ Compare with previous/next elements correctly

Problem 5: Search Range

Problem Statement

Find first and last position of target in sorted array.
Example: `[5,7,7,8,8,10]`, target=8 → `[3,4]`

Approach: Binary Search (Optimal)

Idea: Two binary searches - one for first, one for last.

```java
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int[] ans = {-1, -1};
        if(nums == null || nums.length == 0) return ans;

        // Find first occurrence
        int left = 0, right = nums.length - 1;
        while(left < right) {
            int mid = left + (right - left) / 2;
            if(target <= nums[mid]) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        if(nums[left] != target) return ans;
        ans[0] = left;

        // Find last occurrence
        left = 0;
        right = nums.length - 1;
        while(left < right) {
            int mid = left + (right - left + 1) / 2; // +1!
            if(target >= nums[mid]) {
                left = mid;
            } else {
                right = mid - 1;
            }
        }
        ans[1] = left;
        return ans;
    }
}
```

Time: O(log n) - two binary searches
Space: O(1) - constant variables

Binary Search Variations

First occurrence:
- `right = mid` (keep mid in range)
- Regular mid calculation

Last occurrence:
- `left = mid` (keep mid in range)
- `mid = left + (right - left + 1) / 2` (avoid infinite loop)

Problem 6: Merge Intervals

Problem Statement

Merge all overlapping intervals.
Example: `[[1,3],[2,6],[8,10]]` → `[[1,6],[8,10]]`

Approach: Sort + Merge

Idea: Sort by start time, merge overlapping intervals.

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        if(intervals.length <= 1) return intervals;

        // Sort by start time
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

        List<int[]> result = new ArrayList<>();
        result.add(intervals[0]);

        for(int i = 1; i < intervals.length; i++) {
            int[] last = result.get(result.size() - 1);
            int[] current = intervals[i];

            if(current[0] <= last[1]) {
                // Merge: extend end time
                last[1] = Math.max(last[1], current[1]);
            } else {
                // No overlap: add new interval
                result.add(current);
            }
        }

        return result.toArray(new int[result.size()][]);
    }
}
```

Time: O(n log n) - dominated by sorting
Space: O(n) - result list

Common Mistakes

× Forgetting to use `Math.max()` when merging
× Not sorting before merging
✓ Modify reference directly (no need to update list)

Key Insight

When you get a reference from a list, modifying it directly updates the list:

```java
int[] last = result.get(result.size() - 1);
last[1] = newValue;  // Updates list automatically!
```

Problem 7: Rotate String

Problem Statement

Check if `goal` is a rotation of `s`.
Example: s=`"abcde"`, goal=`"cdeab"` → true

Approach: String Concatenation (Optimal)

Idea: All rotations of s appear as substrings in s+s.

```java
class Solution {
    public boolean rotateString(String s, String goal) {
        return s.length() == goal.length() &&
               (s + s).contains(goal);
    }
}
```

Time: O(n) - contains() uses KMP internally
Space: O(n) - concatenated string

Why s+s Works

`s = "abcde"`
`s + s = "abcdeabcde"`

All rotations appear:
- `"abcde"` at index 0
- `"bcdea"` at index 1
- `"cdeab"` at index 2
- `"deabc"` at index 3
- `"eabcd"` at index 4

Problem 8: Top K Frequent Elements

Problem Statement

Return k most frequent elements.
Example: `[1,1,1,2,2,3]`, k=2 → `[1,2]`

Approach 1: Bucket Sort (Optimal)

Idea: Use frequency as bucket index.

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        // Count frequencies
        Map<Integer, Integer> count = new HashMap<>();
        for(int num : nums) {
            count.put(num, count.getOrDefault(num, 0) + 1)
                ;
        }

        // Create buckets (index = frequency)
        List<Integer>[] buckets = new List[nums.length +
            1];
        for(int i = 0; i <= nums.length; i++) {
            buckets[i] = new ArrayList<>();
        }

        // Fill buckets
        for(int num : count.keySet()) {
            int freq = count.get(num);
            buckets[freq].add(num);
        }

        // Collect top k from highest frequency
        int[] result = new int[k];
        int index = 0;
        for(int freq = buckets.length - 1;
            freq >= 0 && index < k; freq--) {
            for(int num : buckets[freq]) {
                result[index++] = num;
                if(index == k) return result;
            }
        }
        return result;
    }
}
```

Time: O(n) - optimal!
Space: O(n) - buckets array

Approach 2: Min Heap

Idea: Keep heap of size k with smallest frequencies.

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        for(int num : nums) {
            count.put(num, count.getOrDefault(num, 0) + 1)
                ;
        }

        // Min heap of size k
        PriorityQueue<Map.Entry<Integer, Integer>> minHeap
            =
            new PriorityQueue<>((a, b) -> a.getValue() - b
                .getValue());

        for(Map.Entry<Integer, Integer> entry : count.
            entrySet()) {
            minHeap.offer(entry);
            if(minHeap.size() > k) {
                minHeap.poll();  // Remove least frequent
            }
        }

        int[] result = new int[k];
        for(int i = 0; i < k; i++) {
            result[i] = minHeap.poll().getKey();
        }
        return result;
    }
}
```

Time: O(n log k) - better when $k \ll n$
Space: O(n + k) - HashMap + heap

Approach 3: Max Heap (Simpler)

Idea: Add all to max heap, extract top k.

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        for(int num : nums) {
            count.put(num, count.getOrDefault(num, 0) + 1)
                ;
        }

        // Max heap
        PriorityQueue<Map.Entry<Integer, Integer>> maxHeap
            =
            new PriorityQueue<>((a, b) -> b.getValue() - a
                .getValue());

        maxHeap.addAll(count.entrySet());

        int[] result = new int[k];
        for(int i = 0; i < k; i++) {
            result[i] = maxHeap.poll().getKey();
        }
        return result;
    }
}
```

Time: O(n log n) - heap operations
Space: O(n) - heap stores all elements

Which Approach to Use?

Bucket Sort: O(n) - best overall
Min Heap: O(n log k) - when $k \ll n$
Max Heap: O(n log n) - simplest to code

Problem 9: Group Anagrams

Problem Statement

Group strings that are anagrams of each other.
Example: `["eat","tea","tan"]` → `[["eat","tea"],["tan"]]`

Approach: HashMap with Sorted Key

Idea: Use sorted string as key to group anagrams.

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs)
        {
        Map<String, List<String>> map = new HashMap<>();

        for(String s : strs) {
            // Sort string to use as key
            char[] charArray = s.toCharArray();
            Arrays.sort(charArray);
            String sortedS = new String(charArray);

            // Add to group
            map.putIfAbsent(sortedS, new ArrayList<>());
            map.get(sortedS).add(s);
        }

        return new ArrayList<>(map.values());
    }
}
```

Time: O(n · k log k) where k = max string length
Space: O(n · k) - HashMap storage

Alternative: Character Count

Instead of sorting, use character frequency as key:

```
int[] count = new int[26];
for(char c : s.toCharArray()) {
    count[c - 'a']++;
}
String key = Arrays.toString(count);
```

Time: O(n · k) - faster!

3. Code Templates

Two Pointer Template

```java
// Pattern 1: Opposite ends
int left = 0, right = n - 1;
while(left < right) {
    if(condition) {
        // Process and move both
        left++;
        right--;
    } else if(needIncrease) {
        left++;
    } else {
        right--;
    }
}

// Pattern 2: Same direction (slow/fast)
int slow = 0, fast = 0;
while(fast < n) {
    if(condition) {
        // Process
        slow++;
    }
    fast++;
}
```

HashMap Frequency Template

```java
// Count frequencies
Map<T, Integer> freq = new HashMap<>();
for(T item : items) {
    freq.put(item, freq.getOrDefault(item, 0) + 1);
}

// Group by key
Map<Key, List<Value>> groups = new HashMap<>();
for(Value v : values) {
    Key k = getKey(v);
    groups.putIfAbsent(k, new ArrayList<>());
    groups.get(k).add(v);
}
```

Min Heap Template

```java
// Top K pattern (keep k largest)
PriorityQueue<Integer> minHeap = new PriorityQueue
    <>();
for(int num : nums) {
    minHeap.offer(num);
    if(minHeap.size() > k) {
        minHeap.poll();  // Remove smallest
    }
}

// Custom comparator (compare by value)
PriorityQueue<Map.Entry<K,V>> heap =
    new PriorityQueue<>((a,b) -> a.getValue() - b.
        getValue());
```

Binary Search Template

```java
// Standard binary search
int binarySearch(int[] arr, int target) {
    int l = 0, r = arr.length - 1;
    while(l <= r) {
        int mid = l + (r - l) / 2;
        if(arr[mid] == target) return mid;
        else if(arr[mid] < target) l = mid + 1;
        else r = mid - 1;
    }
    return -1;
}

// Find boundary (leftmost/rightmost)
// See Binary Search Pattern in Core Concepts
```

Merge Intervals Template

```java
// Sort by start time
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

List<int[]> result = new ArrayList<>();
result.add(intervals[0]);

for(int i = 1; i < intervals.length; i++) {
    int[] last = result.get(result.size() - 1);
    int[] curr = intervals[i];

    if(curr[0] <= last[1]) {
        // Overlap: merge
        last[1] = Math.max(last[1], curr[1]);
    } else {
        // No overlap: add new
        result.add(curr);
    }
}
```

4. Complexity Analysis

Time Complexity Reasoning

O(1): Direct access, arithmetic operations
`arr[i], map.get(key), x + y`

O(log n): Binary search, heap operations
`Arrays.binarySearch(), heap.offer()`

O(n): Single loop, linear scan
`for(int x : arr), Arrays.stream()`

O(n log n): Sorting, heap with n elements
`Arrays.sort(), Collections.sort()`

O(n^2): Nested loops
`for(i) for(j) ...`

O(n · k): Loop with inner O(k) operation
`for(String s : strs) s.toCharArray()`

Space Complexity Reasoning

O(1): Fixed number of variables
`int x, y, z;`

O(log n): Recursion stack for binary search
`Recursive divide-and-conquer`

O(n): Array/List/Map with n elements
`int[] arr = new int[n], HashMap`

O(n + k): Two structures
`HashMap (n) + PriorityQueue (k)`

What doesn't count:

- Input array (given to you)
- Output array (required by problem)

Common Complexity Mistakes

× Thinking HashSet is O(1) space when it stores n elements (it's O(n))
× Forgetting sorting space (O(log n) for recursion stack)
× Counting output space (usually doesn't count)
✓ Always specify what n, k represent

5. Common Mistakes & Tips

Array Index Errors

× `for(i = 0; i <= n; i++)` - off by one
✓ `for(i = 0; i < n; i++)`

× `arr[arr.length]` - out of bounds
✓ `arr[arr.length - 1]` - last element

× `arr.get(arr.size())` - out of bounds
✓ `arr.get(arr.size() - 1)` - last element

Two Pointer Mistakes

× Not handling duplicates
✓ Skip duplicates after finding match

× Wrong duplicate skip direction
✓ After move: compare with where you came from

× Using `while` instead of `if` for outer loop
✓ Use `if` to skip, not `while`

Binary Search Mistakes

× Infinite loop with `left = mid`
✓ Use `mid = left + (right - left + 1) / 2`

× Not checking final position
✓ Verify `arr[left] == target` after loop

× Overflow: `mid = (left + right) / 2`
✓ Use `mid = left + (right - left) / 2`

HashMap Mistakes

× `map.get(key)` without checking existence
✓ Use `getOrDefault()` or check with `containsKey()`

× Modifying map while iterating
✓ Use `Iterator` or collect keys first

× Forgetting to initialize List in Map
✓ Use `putIfAbsent(key, new ArrayList<>())`

Heap Mistakes

× Using max heap when you need min heap
✓ Default PriorityQueue is min heap

× Wrong comparator direction
✓ `(a,b) -> a-b` is min heap, `(a,b) -> b-a` is max heap

× Not maintaining heap size for Top K
✓ Use `if(heap.size() > k) heap.poll()`

6. Key Insights

When to Use Each Technique

Two Pointer:
- Sorted array
- Finding pairs/triplets
- Palindrome checking
- Remove duplicates in-place

Binary Search:
- Sorted array
- Search/find operations
- Finding boundaries
- Minimize/maximize with condition

HashMap:
- Frequency counting
- Grouping elements
- O(1) lookup needed
- Complement finding (Two Sum)

Heap:
- Top K elements
- Kth largest/smallest
- Merge K sorted arrays
- Running median

Sorting:
- Enable two pointer
- Group anagrams
- Merge intervals
- Detect duplicates (when space limited)

Optimization Strategies

Time:
- HashSet/HashMap for O(1) lookup
- Two pointers instead of nested loops
- Binary search instead of linear search
- Heap for Top K instead of full sort

Space:
- Reuse input array when possible
- Use two variables instead of array
- In-place operations
- Bit manipulation for boolean arrays

Problem Solving Framework

1. **Understand:**
   - Read carefully, note constraints
   - Work through examples
   - Ask clarifying questions
2. **Plan:**
   - Identify pattern (sorted? pairs? frequency?)
   - Choose technique
   - Consider edge cases
3. **Implement:**
   - Write clean, readable code
   - Use meaningful variable names
   - Add comments for tricky parts
4. **Test:**
   - Example cases
   - Edge cases (empty, single element)
   - Large inputs
5. **Optimize:**
   - Analyze complexity
   - Consider alternatives
   - Discuss trade-offs

7. Quick Reference Tables

Complexity Comparison

Operation	Time	Space
Array access	O(1)	-
Array search (unsorted)	O(n)	O(1)
Array search (sorted)	O(log n)	O(1)
Array sort	O(n log n)	O(log n)
HashMap put/get	O(1) avg	O(n)
HashSet add/contains	O(1) avg	O(n)
Heap offer/poll	O(log n)	O(n)
Heap peek	O(1)	O(n)
Two pointer (sorted)	O(n)	O(1)
Binary search	O(log n)	O(1)

Data Structure Selection

Need	Use
Fast lookup	HashMap/HashSet
Sorted order	TreeMap/TreeSet
Min/Max quickly	PriorityQueue
Both ends access	Deque
Dynamic array	ArrayList
Fixed size	Array

Common Patterns

Problem Type	Pattern
Find pairs	Two Pointer
Search in sorted	Binary Search
Count frequency	HashMap
Top K elements	Heap/Bucket Sort
Merge intervals	Sort + Scan
Group items	HashMap
Sliding window	Two Pointer
Prefix/Suffix	Prefix Sum

8. Learning Progression

Recommended Order

Level 1 - Foundations:

1. Contains Duplicate (HashSet basics)
2. Best Time to Buy & Sell Stock (greedy)
3. Two Sum II (two pointer intro)

Level 2 - Core Techniques:

4. Search Range (binary search variations)
5. Rotate String (string manipulation)
6. Group Anagrams (HashMap with sorting)

Level 3 - Advanced:

7. Three Sum (two pointer + dedup)
8. Merge Intervals (sorting + references)
9. Top K Frequent (heap/bucket sort)

Practice Strategy

1. Master basics first:
Arrays, HashMaps, basic loops

2. Learn one pattern at a time:
Two pointer → Binary search → Heap

3. Solve variations:
After mastering one, solve 3-5 similar problems

4. Focus on understanding:
Don't just memorize - understand WHY

5. Review complexity:
Always analyze time and space

Interview Preparation

Week 1-2: Arrays & HashMaps
Week 3-4: Two Pointers & Binary Search
Week 5-6: Heaps & Sorting
Week 7-8: Mixed problems & optimization

Daily:

- 2-3 new problems
- Review 1-2 old problems
- Explain solutions out loud
- Write complexity analysis

9. Final Tips

Amazon Interview Specifics

DO:

- Use built-in methods (Arrays.sort, etc.)
- Explain your approach before coding
- State time/space complexity
- Test with examples
- Discuss trade-offs

DON'T:

- Jump to code immediately
- Ignore edge cases
- Use unclear variable names
- Forget to ask clarifying questions

Code Quality

Variable Naming:

- `left, right` not `l, r`
- `frequency` not `freq`
- `result` not `res`

Structure:

- Handle edge cases first
- One logical operation per line
- Add comments for tricky parts
- Use helper methods when needed

Good luck with your interviews!
Practice, understand, and optimize!
